

LAB MANUAL

Tool Use and Deterministic Control

Day 2 • one lab in six parts: give the Bookshelf assistant a tool, control when it fires, decide the rest in code

Version 1.0 • September 2026

Environment: Cloud Shell in your sandbox GCP project • Gemini 2.5 Flash on Vertex AI (google-genai) • Cloud Shell • agent.py from the Day 2 kit

Contents

How to use this manual

Lab 1 — Give the service a tool, and control when it fires

Part A — No tools

Part B — The Day 1 data store as a server-side tool

Part C — Define `reserve_book` and watch one round

Part D — Control when it fires

Part E — When selection goes wrong

Part F — Determinism in code

Appendix A — `tools.json`, final

Appendix B — `agent.env`: the Lab 1 switches

Appendix C — The callers and their filter strings

How to use this manual

This is **one lab in six parts**, A to F, run one part at a time right after the chapter of the deck that explains it. The kit is one small service, `agent.py`, that runs a Gemini model on Vertex AI in a loop: it sends your message and the tool definitions, runs whatever function call comes back, sends the result in, and repeats until the model answers in text. You edit **two files** — `tools.json`, the tool definitions the model sees, and `agent.env`, your project ids and the switches — and you run the service. You write no code.

Each part flips one switch or makes one edit, and the printout is what you read. → `model` is what was sent (which tools, which mode); ← `model` is what came back — text, or one `function_call` line per call; ⚙️ `run` is our code running the call, or `blocked:` with the reason; ← `function_response` is what went back to the model. The last line counts: `steps` = model calls, `calls` = tool runs, `cost` = tokens × the prices in `agent.env`.

Commands and outputs are in separate blocks

A block captioned **command** holds exactly one line to paste — the whole line, including any `SWITCH=value` typed in front of `python`. A switch typed in front of the command applies to that one run; a switch edited in `agent.env` stays until you change it back.

The block after it, captioned **example** or **expected output**, is what the run prints. Never paste an output block: it goes to the shell and fails with `syntax error near unexpected token`.

Type a message on one line, inside double quotes, exactly as printed.

Blocks captioned **example** show one complete printout from a run of the kit, so you can see every line before you read your own. The model's own words are shown as `<...>` because they differ from run to run; reservation ids are `<id>`; token counts, costs and timings are `<n>`. The lines that matter — → `model`, `function_call`, ⚙️ `run`, `blocked:`, `steps=` and `calls=` — are printed as the kit prints them. Blocks captioned **kit file** print a file from the kit; blocks captioned **edit** show what to put in a file.

The story for today

Meridian Financial's Bookshelf answered questions yesterday from approved documents only, to the caller who may see them, with the caller's filter string chosen by your code. Today the same assistant gets a tool that changes a record: `reserve_book`, which reserves copies of a document in the library through `bookshelf-api`. That is a different kind of risk. A wrong answer can be read and doubted; a wrong reservation has happened.

So the day is about control. Part A runs the assistant with nothing. Part B attaches yesterday's data store as a tool Google runs inside the request, with the caller's filter. Part C defines `reserve_book` — name, description, schema — and watches one call go round. Part D takes the choice of when to call away from the model and gives it back. Part E breaks the tool list three ways. Part F puts the decisions that have a right answer — are the arguments valid, may this run, who is the caller — in code, and gets the same outcome three times.

Three staff profiles are used throughout, from `callers.json` (printed in Appendix C): **priya** (Priya Nair, Customer Contact agent, cleared for `internal`), **daniel** (Daniel Okafor, Dispute Officer, cleared for `confidential`) and **mlro** (Sarah Whitmore, Deputy MLRO, cleared for `restricted`). `--user` picks one; it decides the filter string the tools use, and the model never chooses it.

Cost. Every run is one or more calls to the model, and the last line prints the cost from the token counts and the price lines in `agent.env`. A day of Lab 1 is a few dozen short calls. Nothing today creates a cloud resource; the Day 1 data store and app stay up for Day 3.

Set-up (once, before Part A)

Open Cloud Shell in your sandbox project. Upload `regai-d2-agent-kit.zip` (the upload button in the Cloud Shell toolbar), then unzip it, copy the example environment file, activate the Day 1 venv and install the two packages the kit needs.

```
COMMAND — PASTE THIS LINE
unzip regai-d2-agent-kit.zip && cd regai-d2-agent-kit
```

```
COMMAND — PASTE THIS LINE
cp agent.env.example agent.env
```

```
COMMAND — PASTE THIS LINE
source ~/regai-venv/bin/activate && pip install -q google-genai requests
```

`agent.env` is one of the two files you edit today. Open it in the Cloud Shell Editor (or `nano agent.env`) and fill in the **first block** only: `PROJECT_ID` (your project **id**), `DATASTORE_ID` and `APP_ID` (the console-generated ids from Day 1 — the Details tab of the data store, App overview for the app — not the names), and `BOOKSHELF_API_URL` (the URL `deploy.sh` printed on Day 0). Leave `LOCATION=global` and `MODEL=gemini-2.5-flash`; every other line is a switch at its Part A value.

```
KIT FILE — AGENT.ENV, FIRST BLOCK, FILLED IN. YOUR IDS AND URL WILL DIFFER
# ---- your project (from Day 0 and Day 1)
PROJECT_ID=my-project
LOCATION=global
# If "global" is refused for the model, use the regional line instead.
# LOCATION=us-central1
MODEL=gemini-2.5-flash
DATASTORE_ID=bookshelf-ds_1758000000000
APP_ID=bookshelf-app_1758000000000
BOOKSHELF_API_URL=https://bookshelf-api-<...>.a.run.app
```

Then one run, to see that the model answers and the header lines are right. `--user` is one of `priya`, `daniel`, `mlro`.

```
COMMAND — PASTE THIS LINE
python agent.py --user daniel "hello"
```

```
EXPECTED OUTPUT
caller: daniel (Daniel Okafor, Dispute Officer, clearance confidential)
model: gemini-2.5-flash (Vertex AI, global)
api: https://bookshelf-api-<...>.a.run.app
message: hello
→ model turn 1 tools=[] mode=AUTO grounding=off
← model text: <the model's greeting>
steps=1 calls=0 tokens=<n> cost=$<n> elapsed=<n>s
```

Four header lines on every run: the caller and their clearance, the model and location, the API URL, and your message. `tools=[]` is right for now — `TOOLS=off` is the Part A state. `python agent.py --reset` puts the library back to three copies of every document; the parts say when to run it.

The whole day, in order — every step is explained in its part:

```
THE ORDER
Part A agent.env as shipped (TOOLS=off, GROUNDING=off); the day's question; tools=[]
Part B agent.env: GROUNDING=on; the question as daniel and as priya; then GROUNDING=off
Part C agent.env: TOOLS=on; tools.json: add reserve_book; one reservation, one lookup
Part D MODE=NONE · MODE=ANY · MODE=ANY ALLOWED=reserve_book · two documents · PARALLEL=off
Part E tools.json: "does things" · five look-alikes · fixed — the same request three times
Part F VALIDATE=on · APPROVE=off three times · --user priya reserving for daniel
```

LAB 1 · DIFFICULTY ●●●

Lab 1 — Give the service a tool, and control when it fires

Objective

Give the Bookshelf assistant a tool that changes a record, and control when it fires and what happens when it does — from two files and a printout. By the end of Part F you have: the assistant run with no tools; the Day 1 data store attached as a server-side tool with the caller's filter; `reserve_book` defined in `tools.json` and one function call watched all the way round; the model's choice taken away (NONE), forced (ANY, ALLOWED) and split (PARALLEL=off); the tool list broken by a bad description and by five look-alike names and fixed again; and three decisions — valid arguments, may this run, who is the caller — made in code, with the same request giving the same outcome three times.

Environment

Cloud Shell in your own sandbox project, with the Day 1 venv (`~/regai-venv`) active, the kit `regai-d2-agent-kit.zip` unzipped and `agent.env` filled in. Everything today is `python agent.py ...` from the kit folder; the two files you edit are `tools.json` and `agent.env`. The Day 1 data store and app, and the Day 0 `bookshelf-api` on Cloud Run, must still be up.

Prerequisites

- Day 1 finished: the data store and app are up, and you have their console-generated ids
- Day 0's `bookshelf-api` is deployed on Cloud Run, and your account can call it (`roles/run.invoker`)
- The set-up above done: kit unzipped, `agent.env` first block filled in, venv active, the `hello` run printed its four header lines
- The deck's chapter for each part taught before you start that part — each part assumes its chapter

The six parts

A — No tools. `agent.env` as shipped. The day's question; the model has nothing to answer from.

B — A server-side tool. `GROUNDING=on`: the Day 1 data store, with the caller's filter string. `daniel` gets a cited answer; `priya` does not. No  `run` line — the retrieval ran on Google's side.

C — Define a tool. `TOOLS=on`, and `reserve_book` added to `tools.json`. One reservation: `function_call` →  `run` → `function_response` → confirmation. One stock question.

D — Control when it fires. `MODE=NONE`, `MODE=ANY`, `ALLOWED=reserve_book`, two documents in one request, `PARALLEL=off`. Five commands, no edits.

E — When selection goes wrong. A description that says nothing; five look-alike names; the fixed list. The same request three times.

F — Determinism in code. `VALIDATE=on` refuses bad arguments before any call; `APPROVE=off` blocks the reservation, three identical times; `--user priya` cannot reserve for `daniel`.

Part A — No tools

Goal

Run the assistant as shipped — `TOOLS=off`, `GROUNDING=off` — and ask it the day's question. The `→ model` line says `tools=[]`: nothing was offered. The model answers from nothing, or says it cannot. Nothing to edit in this part.

Steps

1. Check the two switches in `agent.env` are at their shipped values. Nothing else in the file matters yet.

COMMAND — PASTE THIS LINE

```
grep -E '^(TOOLS|GROUNDING)= ' agent.env
```

EXPECTED OUTPUT

```
TOOLS=off
GROUNDING=off
```

2. Ask the day's question as daniel. One line; the question inside double quotes.

COMMAND — PASTE THIS LINE

```
python agent.py --user daniel "Which form must a cardholder sign before a dispute can go to chargeback?"
```

EXAMPLE

```
caller: daniel (Daniel Okafor, Dispute Officer, clearance confidential)
model: gemini-2.5-flash (Vertex AI, global)
api: <your BOOKSHELF_API_URL>
message: Which form must a cardholder sign before a dispute can go to chargeback?
→ model turn 1 tools=[] mode=AUTO grounding=off
→ model text: <the model's reply — it should say it cannot tell you, or refuse; write down what yours says>
steps=1 calls=0 tokens=<n> cost=$<n> elapsed=<n>s
```

3. Read the `→ model` line: `tools=[]`. Nothing was sent that the model could call. Then read the `← model` line and write down two things: what the model said, and what it would have needed in order to answer. If it named a form, write that down too — it had no document to take it from.

✓ Check yourself

The four header lines show your caller, `gemini-2.5-flash` (Vertex AI, `global`) (or `us-central1`), your API URL and your message.

`→ model turn 1 tools=[] mode=AUTO grounding=off` — an empty tool list.

`steps=1 calls=0`: one model call, no tool run. There is no  `run` line anywhere.

You wrote down what the model said and what it was missing.

Troubleshooting

Symptom	Cause	Fix
<code>agent.env</code> not found	The set-up's <code>cp</code> was skipped	<code>cp agent.env.example agent.env</code> , then fill in the first block
<code>ModuleNotFoundError: No module named 'google'</code>	The venv is not active, or <code>pip install</code> was skipped	<code>source ~/regai-venv/bin/activate && pip install -q google-genai requests</code>
An error about the model or location on the first call	<code>LOCATION=global</code> is refused for the model in your project	In <code>agent.env</code> comment out <code>LOCATION=global</code> and uncomment <code>LOCATION=us-central1</code> ; rerun
The model answered with a form number	It answered from training, with no document	Correct behaviour for Part A — that is the point; Part B gives it documents

Part B — The Day 1 data store as a server-side tool

Goal

One switch: `GROUNDING=on`. `agent.py` adds the Day 1 data store to the request as a retrieval tool, with the caller's filter string from `callers.json` — the same string `answer.sh --user` sent yesterday. Ask the question as daniel: a cited answer, `grounding:` lines, and no  `run` line, because the retrieval ran on Google's side. Ask it as priya: her filter stops at `internal`, the dispute policy is confidential, and the answer says so. Then set the switch back for Part C.

Steps

1. In `agent.env` set `GROUNDING=on`. Leave `TOOLS=off`.

EDIT — AGENT.ENV

```
GROUNDING=on
```

2. Read the two filter strings the tool will carry. They are the ones from Day 1; `--user` picks one and the model never sees it.

COMMAND — PASTE THIS LINE

```
python3 -c "import json; c=json.load(open('callers.json')); [print(u, '=>', c[u]['filter']) for u in ('daniel','priya')]"
```

EXPECTED OUTPUT

```
daniel => status: ANY("approved") AND classification: ANY("internal","confidential")
priya => status: ANY("approved") AND classification: ANY("internal")
```

3. Ask as daniel.

COMMAND — PASTE THIS LINE

```
python agent.py --user daniel "Which form must a cardholder sign before a dispute can go to chargeback?"
```

EXAMPLE

```
caller: daniel (Daniel Okafor, Dispute Officer, clearance confidential)
model: gemini-2.5-flash (Vertex AI, global)
api: <your BOOKSHELF_API_URL>
message: Which form must a cardholder sign before a dispute can go to chargeback?
→ model turn 1 tools=[] mode=AUTO grounding=on
- model text: <the model's answer – expect it to name form FRM-CD-7, from the dispute policy>
  grounding: [1] Credit Card Dispute and Chargeback Handling Policy (gs://<your
    bucket>/corpus-html/POL-CR-014.html)
  grounding: server-side retrieval – no call from our side
steps=1 calls=0 tokens=<n> cost=$<n> elapsed=<n>s
```

Three lines to read. `grounding=on` on the `→ model` line, and `tools=[]` still — the retrieval tool is not a function declaration, so it is not in that list. One `grounding:` line per retrieved chunk, naming the document and its uri in your bucket. Then `server-side retrieval – no call from our side`: there is no  `run` line, because nothing ran here. `steps=1 calls=0`.

4. Ask as priya.

COMMAND — PASTE THIS LINE

```
python agent.py --user priya "Which form must a cardholder sign before a dispute can go to chargeback?"
```

EXAMPLE

```
caller: priya (Priya Nair, Customer Contact agent, clearance internal)
model: gemini-2.5-flash (Vertex AI, global)
api: <your BOOKSHELF_API_URL>
message: Which form must a cardholder sign before a dispute can go to chargeback?
→ model turn 1 tools=[] mode=AUTO grounding=on
- model text: <the refusal – "The approved documents I can see do not answer this." or the model's own
  words for it>
  grounding: no chunks returned (nothing passed the caller's filter)
steps=1 calls=0 tokens=<n> cost=$<n> elapsed=<n>s
```

Same question, same data store, one minute apart. priya's filter string excludes confidential documents, the dispute policy is confidential, so the retrieval passed nothing and the model had nothing to answer from — `grounding: no chunks returned (nothing passed the caller's filter)`. The string was chosen by `agent.py` from `--user`; nothing priya typed could change it.

5. Set `GROUNDING=off` again in `agent.env` before Part C, or every Part C printout carries a retrieval it does not need.

EDIT — AGENT.ENV

```
GROUNDING=off
```

✓ Check yourself

daniel: `grounding=on`, a `grounding: [1] ...` line naming the dispute policy, no call from our side, no ⚙️ run.

priya: `grounding: no chunks returned`, and a refusal.

You can say where the filter string came from — `callers.json`, chosen by `--user` — and where it went: inside the retrieval tool, in the request.

`GROUNDING=off` again.

Troubleshooting

Symptom	Cause	Fix
An error naming the data store (not found, 404)	<code>DATASTORE_ID</code> in <code>agent.env</code> is the display name, not the console-generated id	Data Stores → <code>bookshelf-ds</code> → Details → the ID field; put it in <code>agent.env</code>
403 mentioning Discovery Engine	The Discovery Engine API is not enabled, or your role lacks it	Day 1 Part A; ask the project owner
daniel's answer names FRM-CD-6	The superseded policy passed the filter	A Day 1 schema problem — <code>status</code> is not indexable; see Day 1 Part B
<code>grounding:</code> lines appear but the answer ignores them	The model's choice	Write it down; the deck's chapter 2 debrief takes it
Part C's printout says <code>grounding=on</code>	Step 5 skipped	<code>GROUNDING=off</code> in <code>agent.env</code>

Part C — Define `reserve_book` and watch one round

Goal

Two switches and one edit. `TOOLS=on` sends the tools in `tools.json` to the model; `GROUNDING=off`. Then add `reserve_book` to `tools.json` — the name, the description, and the schema: `book_id` a string, `copies` an integer from 1 to 3, `caller` one of the three ids. Run one reservation and read the four lines of a tool round: `function_call`, `run`, `function_response`, the confirmation. `lookup_stock` was already there; one stock question shows one copy left.

Steps

1. In `agent.env` set `TOOLS=on` and make sure `GROUNDING=off`.

EDIT — AGENT.ENV

```
TOOLS=on
GROUNDING=off
```

2. Read `tools.json` as shipped: two tools, `lookup_stock` and `search_bookshelf`, each with a name, a description and a JSON Schema under `parameters`. You are about to add a third.

COMMAND — PASTE THIS LINE

```
python3 -c "import json; [print(t['name'], '-', t['description'][:60]) for t in json.load(open('tools.json'))]"
```

EXPECTED OUTPUT

```
lookup_stock - Read-only: how many copies of one document in the Bookshelf
search_bookshelf - Search the approved policies and procedures and answer a que
```

3. Add `reserve_book` to `tools.json`, after `search_bookshelf`'s entry and before the closing `]`. Open the file in the Cloud Shell Editor, put a comma after the `}` that closes `search_bookshelf`, and paste the object below. Appendix A prints the whole finished file; `fixtures/tools.lab1c.json` is the same file if you would rather copy it (`cp fixtures/tools.lab1c.json tools.json`).

EDIT — TOOLS.JSON: THE ENTRY TO ADD (MIND THE COMMA BEFORE IT). THE DESCRIPTION IS ONE LINE

```
{
  "name": "reserve_book",
  "description": "Reserve copies of a document in the Bookshelf library for a member of staff. Use when the caller asks to reserve, hold or book copies. This changes library records.",
  "parameters": {
    "type": "object",
    "properties": {
      "book_id": {"type": "string", "description": "Document id, e.g. PRC-KYC-003"},
      "copies": {"type": "integer", "minimum": 1, "maximum": 3, "description": "Copies to reserve, 1 to 3"},
      "caller": {"type": "string", "enum": ["priya", "daniel", "mlro"], "description": "Who the reservation is for"}
    },
    "required": ["book_id", "copies", "caller"]
  }
}
```

Three parts. The **name** is what the model writes in its `function_call` and what `agent.py` looks up in `tools_impl.py` — `reserve_book`, exactly. The **description** says what it does, when to use it in the caller's words ("reserve, hold or book"), and what it changes. The **parameters** are a JSON Schema: `book_id` is a string; `copies` is an integer with `minimum` 1 and `maximum` 3; `caller` is an `enum` of the three ids; all three are required.

4. Check the file parses, and put the library at three copies of everything.

COMMAND — PASTE THIS LINE

```
python3 -c "import json; print([t['name'] for t in json.load(open('tools.json'))])"
```

EXPECTED OUTPUT

```
['lookup_stock', 'search_bookshelf', 'reserve_book']
```

COMMAND — PASTE THIS LINE

```
python agent.py --reset
```

EXPECTED OUTPUT

```
{"status": "reset", "books": 13}
```

5. Reserve two copies.**COMMAND — PASTE THIS LINE**

```
python agent.py --user daniel "Reserve two copies of PRC-KYC-003 for daniel"
```

EXAMPLE

```
caller: daniel (Daniel Okafor, Dispute Officer, clearance confidential)
model: gemini-2.5-flash (Vertex AI, global)
api: <your BOOKSHELF_API_URL>
message: Reserve two copies of PRC-KYC-003 for daniel
→ model turn 1 tools=[lookup_stock, search_bookshelf, reserve_book] mode=AUTO grounding=off
← model function_call reserve_book({"book_id": "PRC-KYC-003", "copies": 2, "caller": "daniel"})
⚙️ run reserve_book({"book_id": "PRC-KYC-003", "copies": 2, "caller": "daniel"}) → {"reservation_id": "<id>",
"book_id": "PRC-KYC-003", "copies": 2, "copies_available": 1, "replayed": false}
← function_response reserve_book {"reservation_id": "<id>", "book_id": "PRC-KYC-003", "copies": 2,
"copies_available": 1, "replayed": false}
→ model turn 2 tools=[lookup_stock, search_bookshelf, reserve_book] mode=AUTO grounding=off
← model text: <the confirmation — the reservation, and how many copies are left>
steps=2 calls=1 tokens=<n> cost=$<n> elapsed=<n>s
```

Four lines to read, in order. `← model function_call reserve_book({...})` — the model's turn: the tool by name and the arguments in the schema's shape. `⚙️ run reserve_book(...) → {...}` — our code ran it against `bookshelf-api` and got a `reservation_id` and `copies_available`. `← function_response reserve_book {...}` — the same result, sent back to the model. Then turn 2 and `← model text:` — the model reports it. `steps=2 calls=1`: two model calls, one tool run.

6. Ask how many are left. `lookup_stock` was in the file all along.**COMMAND — PASTE THIS LINE**

```
python agent.py --user daniel "How many copies of PRC-KYC-003 are left?"
```

EXAMPLE

```
caller: daniel (Daniel Okafor, Dispute Officer, clearance confidential)
model: gemini-2.5-flash (Vertex AI, global)
api: <your BOOKSHELF_API_URL>
message: How many copies of PRC-KYC-003 are left?
→ model turn 1 tools=[lookup_stock, search_bookshelf, reserve_book] mode=AUTO grounding=off
← model function_call lookup_stock({"book_id": "PRC-KYC-003"})
⚙️ run lookup_stock({"book_id": "PRC-KYC-003"}) → {"book_id": "PRC-KYC-003", "title": "Customer
Identification and Verification Procedure", "copies_available": 1, "copies_total": 3}
← function_response lookup_stock {"book_id": "PRC-KYC-003", "title": "Customer Identification and
Verification Procedure", "copies_available": 1, "copies_total": 3}
→ model turn 2 tools=[lookup_stock, search_bookshelf, reserve_book] mode=AUTO grounding=off
← model text: <the answer — 1 of 3 copies>
steps=2 calls=1 tokens=<n> cost=$<n> elapsed=<n>s
```

✓ Check yourself

`→ model turn 1 tools=[lookup_stock, search_bookshelf, reserve_book]` — three names.

The four lines of the round, in order: `function_call` · `⚙️ run` · `function_response` · `text`. The name is the same on all of them.

`copies_available` went from 3 to 1 in the `⚙️ run` line, and `lookup_stock` says 1 of 3. `steps=2 calls=1` on both runs.

Troubleshooting

Symptom	Cause	Fix
<code>json.decoder.JSONDecode Error</code>	A missing comma before the new entry, or a trailing comma after it	Fix the comma, or <code>cp fixtures/tools.lab1c.json tools.json</code>
<code>blocked: no implementation for 'reserve_books'</code>	The name was typed with an <code>S</code> , or otherwise differs from <code>reserve_book</code>	Rename it back — <code>tools_impl.py</code> knows <code>reserve_book</code>
<code>HTTP 401 / HTTP 403</code> from <code>bookshelf-api</code>	No identity token, or your account lacks <code>roles/run.invoker</code>	<code>gcloud auth login</code> ; ask for the role on the service
<code>BOOKSHELF_API_URL</code> is empty	The set-up's first block is incomplete	The URL <code>deploy.sh</code> printed on Day 0, into <code>agent.env</code>
<code>HTTP 409 ... only n copies available</code>	The library is low from a previous run	<code>python agent.py --reset</code>
<code>tools=[]</code> on the <code>→ model</code> line	<code>TOOLS=off</code> still	Step 1
The model called <code>lookup_stock</code> first, then <code>reserve_book</code>	Its choice: check, then act	Fine — <code>steps=3 calls=2</code> ; read all the lines

Part D — Control when it fires

Goal

Five commands, no edits. Each switch is typed in front of `python` and applies to that run only. `MODE=NONE` on a reservation request: talk only. `MODE=ANY` on "hello": a call the request never needed — read the last line for what it cost. `MODE=ANY ALLOWED=reserve_book` on "hello": a forced reservation. Two documents in one request: two `function_call` lines in one turn. `PARALLEL=off` on the same request: the second call is refused and comes back in the next turn.

Steps

1. `MODE=NONE`: the tools are still sent, and the model may not call any of them.

COMMAND — PASTE THIS LINE

```
MODE=NONE python agent.py --user daniel "Reserve one copy of PRC-KYC-003 for daniel"
```

EXAMPLE

```
caller: daniel (Daniel Okafor, Dispute Officer, clearance confidential)
model: gemini-2.5-flash (Vertex AI, global)
api: <your BOOKSHELF_API_URL>
message: Reserve one copy of PRC-KYC-003 for daniel
→ model turn 1 tools=[lookup_stock, search_bookshelf, reserve_book] mode=NONE grounding=off
← model text: <the model's reply — it should say it cannot act, or ask; no reservation>
steps=1 calls=0 tokens=<n> cost=$<n> elapsed=<n>s
```

`mode=NONE` on the `→ model` line is what was sent, and the API enforces it. No ⚙️ run line; `steps=1 calls=0`.

2. `MODE=ANY` on "hello": the model must return a function call, so it does.

COMMAND — PASTE THIS LINE

```
MODE=ANY python agent.py --user daniel "hello"
```

EXAMPLE

```
caller: daniel (Daniel Okafor, Dispute Officer, clearance confidential)
model: gemini-2.5-flash (Vertex AI, global)
api: <your BOOKSHELF_API_URL>
message: hello
→ model turn 1 tools=[lookup_stock, search_bookshelf, reserve_book] mode=ANY grounding=off
← model function_call <a tool the model chose>({<arguments it invented>})
⚙️ run <a tool the model chose>({<arguments it invented>}) → {"book_id": "POL-CR-014", "title": "Credit Card
    Dispute and Chargeback Handling Policy", "copies_available": 3, "copies_total": 3}
← function_response <that tool> {...}
→ model turn 2 tools=[lookup_stock, search_bookshelf, reserve_book] mode=AUTO (ANY applied to the first
    turn only) grounding=off
← model text: <the greeting, plus whatever it says about the call it was made to do>
steps=2 calls=1 tokens=<n> cost=$<n> elapsed=<n>s
```

Which tool, and with what argument, is the model's choice — there was nothing in "hello" to choose from. Read the last line: `steps=2 calls=1` — two model turns and a tool run for a greeting. On turn 2 the `→ model` line says `mode=AUTO` (ANY applied to the first turn only): `agent.py` forces a call on the first turn only, because a loop forced on every turn could never answer in text.

3. ALLOWED=reserve_book: the forced call must be that tool. A reservation for a greeting.

COMMAND — PASTE THIS LINE

```
MODE=ANY ALLOWED=reserve_book python agent.py --user daniel "hello"
```

EXAMPLE

```
caller: daniel (Daniel Okafor, Dispute Officer, clearance confidential)
model: gemini-2.5-flash (Vertex AI, global)
api: <your BOOKSHELF_API_URL>
message: hello
→ model turn 1 tools=[lookup_stock, search_bookshelf, reserve_book] mode=ANY allowed=[reserve_book]
  grounding=off
← model function_call reserve_book({"book_id": "<a document the model chose>", "copies": <n>, "caller":
  "daniel"})
⚙️ run reserve_book(...) → {"reservation_id": "<id>", ...}
← function_response reserve_book {...}
→ model turn 2 tools=[lookup_stock, search_bookshelf, reserve_book] mode=AUTO (ANY applied to the first
  turn only) grounding=off
← model text: <the greeting, plus what it says about the reservation it was made to do>
steps=2 calls=1 tokens=<n> cost=$<n> elapsed=<n>s
```

allowed=[reserve_book] on the → model line. A record changed, on a document the model picked, because the request said a call was required. Put the library back before the next step.

COMMAND — PASTE THIS LINE

```
python agent.py --reset
```

4. Two documents in one request. Read the ← model lines before any ⚙️ run.

COMMAND — PASTE THIS LINE

```
python agent.py --user daniel "Reserve one copy of PRC-KYC-003 and PRC-CR-030 for daniel"
```

EXAMPLE

```
caller: daniel (Daniel Okafor, Dispute Officer, clearance confidential)
model: gemini-2.5-flash (Vertex AI, global)
api: <your BOOKSHELF_API_URL>
message: Reserve one copy of PRC-KYC-003 and PRC-CR-030 for daniel
→ model turn 1 tools=[lookup_stock, search_bookshelf, reserve_book] mode=AUTO grounding=off
← model function_call reserve_book({"book_id": "PRC-KYC-003", "copies": 1, "caller": "daniel"})
← model function_call reserve_book({"book_id": "PRC-CR-030", "copies": 1, "caller": "daniel"})
⚙️ run reserve_book({"book_id": "PRC-KYC-003", "copies": 1, "caller": "daniel"}) → {"reservation_id": "<id>",
  "book_id": "PRC-KYC-003", "copies": 1, "copies_available": 2, "replayed": false}
⚙️ run reserve_book({"book_id": "PRC-CR-030", "copies": 1, "caller": "daniel"}) → {"reservation_id": "<id>",
  "book_id": "PRC-CR-030", "copies": 1, "copies_available": 2, "replayed": false}
← function_response reserve_book {"reservation_id": "<id>", "book_id": "PRC-KYC-003", "copies": 1,
  "copies_available": 2, "replayed": false}
← function_response reserve_book {"reservation_id": "<id>", "book_id": "PRC-CR-030", "copies": 1,
  "copies_available": 2, "replayed": false}
→ model turn 2 tools=[lookup_stock, search_bookshelf, reserve_book] mode=AUTO grounding=off
← model text: <both reservations, confirmed>
steps=2 calls=2 tokens=<n> cost=$<n> elapsed=<n>s
```

Two ← model function_call lines, then two ⚙️ run lines, then two ← function_response lines, then one more turn: steps=2 calls=2. The model asked for both at once, and PARALLEL=on — the shipped value — ran both. If your model returned them one per turn instead, your printout shows steps=3 calls=2 without any switch; note it for the debrief. Reset again, so both documents are back at three copies.

COMMAND — PASTE THIS LINE

```
python agent.py --reset
```

5. PARALLEL=off: run the first call, refuse the rest, let the model ask again.

COMMAND — PASTE THIS LINE

```
PARALLEL=off python agent.py --user daniel "Reserve one copy of PRC-KYC-003 and PRC-CR-030 for daniel"
```

EXAMPLE

```
caller: daniel (Daniel Okafor, Dispute Officer, clearance confidential)
model: gemini-2.5-flash (Vertex AI, global)
api: <your BOOKSHELF_API_URL>
message: Reserve one copy of PRC-KYC-003 and PRC-CR-030 for daniel
→ model turn 1 tools=[lookup_stock, search_bookshelf, reserve_book] mode=AUTO grounding=off
→ model function_call reserve_book({"book_id": "PRC-KYC-003", "copies": 1, "caller": "daniel"})
→ model function_call reserve_book({"book_id": "PRC-CR-030", "copies": 1, "caller": "daniel"})
✱ run reserve_book({"book_id": "PRC-KYC-003", "copies": 1, "caller": "daniel"}) → {"reservation_id": "<id>",
"book_id": "PRC-KYC-003", "copies": 1, "copies_available": 2, "replayed": false}
✱ run reserve_book({"book_id": "PRC-CR-030", "copies": 1, "caller": "daniel"}) → blocked: one action per
turn (PARALLEL=off)
→ function_response reserve_book {"reservation_id": "<id>", "book_id": "PRC-KYC-003", "copies": 1,
"copies_available": 2, "replayed": false}
→ function_response reserve_book {"error": "one action per turn"}
→ model turn 2 tools=[lookup_stock, search_bookshelf, reserve_book] mode=AUTO grounding=off
→ model function_call reserve_book({"book_id": "PRC-CR-030", "copies": 1, "caller": "daniel"})
✱ run reserve_book({"book_id": "PRC-CR-030", "copies": 1, "caller": "daniel"}) → {"reservation_id": "<id>",
"book_id": "PRC-CR-030", "copies": 1, "copies_available": 2, "replayed": false}
→ function_response reserve_book {"reservation_id": "<id>", "book_id": "PRC-CR-030", "copies": 1,
"copies_available": 2, "replayed": false}
→ model turn 3 tools=[lookup_stock, search_bookshelf, reserve_book] mode=AUTO grounding=off
→ model text: <both reservations, confirmed>
steps=3 calls=2 tokens=<n> cost=$<n> elapsed=<n>s
```

The second ✱ run line says `blocked: one action per turn (PARALLEL=off)`, and its `function_response` carries `{"error": "one action per turn"}`. On turn 2 the model asks for the second document again, it runs, and the model answers on turn 3: `steps=3 calls=2`. One more model turn; in exchange, each side effect ran alone, after the model had seen the result of the one before. Gemini has no request setting that stops parallel calls — this is your loop deciding.

✓ Check yourself

Five last lines: `steps=1 calls=0 (NONE) · steps=2 calls=1 (ANY) · steps=2 calls=1` with `allowed=[reserve_book] · steps=2 calls=2 (two calls) · steps=3 calls=2 (PARALLEL=off)`.

You can point at the line that proves no tool could be called under `MODE=NONE`, and say why it is not the same as a sentence in the prompt.

You can name what the forced call on "hello" cost, and which of the two forced runs changed a record.

You can say what the extra turn under `PARALLEL=off` bought.

Troubleshooting

Symptom	Cause	Fix
note: ALLOWED=reserve_book is ignored unless MODE=ANY	ALLOWED typed without MODE=ANY	Type both switches, as printed
HTTP 409 ... only n copies available	The library is low	<code>python agent.py --reset</code> , rerun
Step 4 shows one function_call per turn	The model chose to call them one at a time	Not a fault; <code>steps=3 calls=2</code> without the switch. Keep the printout
Under MODE=ANY the model called search_bookshelf	Its choice among the three	Fine — the point is that a call was forced, not which
mode=AUTO on turn 2 although you typed MODE=ANY	The first-turn rule	Expected — the line says so in brackets

Part E — When selection goes wrong

Goal

Three edits to `tools.json`, the same reservation request each time. Change `reserve_book`'s description to "does things": the model picks the wrong tool. Put it back and add five look-alikes with the same schema and a one-line description each: the model picks one that has no implementation. Delete the five: the reservation runs. The `fixtures` folder holds the two broken files and the fixed one, if you would rather copy than edit.

Steps

1. Put the library back, then in `tools.json` change `reserve_book`'s description to "does things" — the whole string, nothing else. Or copy the broken file.

COMMAND — PASTE THIS LINE

```
python agent.py --reset
```

EDIT — TOOLS.JSON, RESERVE_BOOK'S DESCRIPTION LINE

```
"description": "does things",
```

COMMAND — INSTEAD OF THE EDIT, IF YOU PREFER

```
cp fixtures/tools.lab1e-bad.json tools.json
```

2. The reservation from Part C. Which tool ran?

COMMAND — PASTE THIS LINE

```
python agent.py --user daniel "Reserve two copies of PRC-KYC-003 for daniel"
```

EXAMPLE

```
caller: daniel (Daniel Okafor, Dispute Officer, clearance confidential)
model: gemini-2.5-flash (Vertex AI, global)
api: <your BOOKSHELF_API_URL>
message: Reserve two copies of PRC-KYC-003 for daniel
→ model turn 1 tools=[lookup_stock, search_bookshelf, reserve_book] mode=AUTO grounding=off
→ model function_call lookup_stock({"book_id": "PRC-KYC-003"}) → not reserve_book
⚙️ run lookup_stock({"book_id": "PRC-KYC-003"}) → {"book_id": "PRC-KYC-003", "title": "Customer
  Identification and Verification Procedure", "copies_available": 3, "copies_total": 3}
→ function_response lookup_stock {"book_id": "PRC-KYC-003", "title": "Customer Identification and
  Verification Procedure", "copies_available": 3, "copies_total": 3}
→ model turn 2 tools=[lookup_stock, search_bookshelf, reserve_book] mode=AUTO grounding=off
→ model text: <whatever the model made of the result — a stock count reported as if it were the answer, or
  a refusal>
steps=2 calls=1 tokens=<n> cost=$<n> elapsed=<n>s
```

The model had the name `reserve_book` and a description that said nothing, and it chose `lookup_stock` — the only tool whose description mentioned copies. Yours may have chosen differently, asked a question, or picked `reserve_book` from its name alone; write down which. Every one of those is the description failing, not the model.

3. Put the description back, then add five look-alikes after `reserve_book`: `reserve_books`, `book_reserve`, `reserve_copy`, `make_reservation`, `reserve_book_for_staff` — each a copy of the `reserve_book` entry with the new name and a one-line description. Or copy the file.

EDIT — TOOLS.JSON, RESERVE_BOOK'S DESCRIPTION LINE, RESTORED (ONE LINE)

```
"description": "Reserve copies of a document in the Bookshelf library for a member of staff. Use when the
  caller asks to reserve, hold or book copies. This changes library records.",
```

COMMAND — INSTEAD OF THE FIVE EDITS, IF YOU PREFER

```
cp fixtures/tools.lab1e-dupes.json tools.json
```

COMMAND — PASTE THIS LINE

```
python3 -c "import json; print([t['name'] for t in json.load(open('tools.json'))])"
```

EXPECTED OUTPUT — EIGHT NAMES

```
['lookup_stock', 'search_bookshelf', 'reserve_book', 'reserve_books', 'book_reserve', 'reserve_copy',
 'make_reservation', 'reserve_book_for_staff']
```

4. The same request. Read the → `model` line, then the ⚙️ `run` line.

COMMAND — PASTE THIS LINE

```
python agent.py --user daniel "Reserve two copies of PRC-KYC-003 for daniel"
```

EXAMPLE

```
caller: daniel (Daniel Okafor, Dispute Officer, clearance confidential)
model: gemini-2.5-flash (Vertex AI, global)
api: <your BOOKSHELF_API_URL>
message: Reserve two copies of PRC-KYC-003 for daniel
→ model turn 1 tools=[lookup_stock, search_bookshelf, reserve_book, reserve_books, book_reserve,
    reserve_copy, make_reservation, <one of the six reserve names>] mode=AUTO grounding=off
→ model function_call <one of the six reserve names>({"book_id": "PRC-KYC-003", "copies": 2, "caller":
    "daniel"})
⚙️ run <one of the six reserve names>({"book_id": "PRC-KYC-003", "copies": 2, "caller": "daniel"}) → blocked:
    no implementation for '<that name>' (declared in tools.json, missing in tools_impl.py)
→ function_response <one of the six reserve names> {"error": "no implementation for <one of the six reserve
    names>"}
→ model turn 2 tools=[lookup_stock, search_bookshelf, reserve_book, reserve_books, book_reserve,
    reserve_copy, make_reservation, <one of the six reserve names>] mode=AUTO grounding=off
→ model text: <the model reporting that it could not reserve>
steps=2 calls=0 tokens=<n> cost=$<n> elapsed=<n>s
```

Eight tools on the → `model` line, on both turns. The model chose one of the six ways to reserve — a reasonable reading of the request — and `agent.py` blocked it: no implementation for '...' (declared in `tools.json`, missing in `tools_impl.py`). `calls=0`. If yours happened to pick `reserve_book`, the reservation ran — and you would not want to rely on that in production. Note the token count on the last line against Part C's: every declaration is in the prompt on every turn.

5. Delete the five look-alikes (or copy the fixed file), check the names, reset, and run once more.

COMMAND — PASTE THIS LINE

```
cp fixtures/tools.lab1e-fixed.json tools.json
```

COMMAND — PASTE THIS LINE

```
python3 -c "import json; print([t['name'] for t in json.load(open('tools.json'))])"
```

EXPECTED OUTPUT

```
['lookup_stock', 'search_bookshelf', 'reserve_book']
```

COMMAND — PASTE THIS LINE

```
python agent.py --reset
```

COMMAND — PASTE THIS LINE

```
python agent.py --user daniel "Reserve two copies of PRC-KYC-003 for daniel"
```

EXAMPLE

```
caller: daniel (Daniel Okafor, Dispute Officer, clearance confidential)
model: gemini-2.5-flash (Vertex AI, global)
api: <your BOOKSHELF_API_URL>
message: Reserve two copies of PRC-KYC-003 for daniel
→ model turn 1 tools=[lookup_stock, search_bookshelf, reserve_book] mode=AUTO grounding=off
→ model function_call reserve_book({"book_id": "PRC-KYC-003", "copies": 2, "caller": "daniel"})
⚙️ run reserve_book({"book_id": "PRC-KYC-003", "copies": 2, "caller": "daniel"}) → {"reservation_id": "<id>",
  "book_id": "PRC-KYC-003", "copies": 2, "copies_available": 1, "replayed": false}
→ function_response reserve_book {"reservation_id": "<id>", "book_id": "PRC-KYC-003", "copies": 2,
  "copies_available": 1, "replayed": false}
→ model turn 2 tools=[lookup_stock, search_bookshelf, reserve_book] mode=AUTO grounding=off
→ model text: <the confirmation>
steps=2 calls=1 tokens=<n> cost=$<n> elapsed=<n>s
```

✓ Check yourself

Run 1: a tool other than `reserve_book` ran, or the model did not reserve — and you wrote down which.

Run 2: eight names on the `→ model` line; a `blocked: no implementation` line; `calls=0`.

Run 3: three names; ⚙️ `run reserve_book(...) → {"reservation_id": ...}`.

You can say, for each run, which part of the list failed: the description, the names, the length.

Troubleshooting

Symptom	Cause	Fix
<code>json.decoder.JSONDecode Error</code>	A comma lost or added while editing	Copy the fixture file for that run
Run 2 reserved anyway	The model picked <code>reserve_book</code> among the six	A coin toss that landed well; the eight-name line is still the fault
Run 3: <code>blocked: no implementation</code>	A look-alike survived the deletion, or <code>reserve_book</code> itself was renamed	<code>cp fixtures/tools.lab1e-fixed.json tools.json</code>
HTTP 409	The library is low	<code>python agent.py --reset</code>

Part F — Determinism in code

Goal

Five commands, no edits. "Five copies" is outside the schema: without `VALIDATE` the call is made and `bookshelf-api` refuses it; with `VALIDATE=on` `agent.py` refuses it before any call. `APPROVE=off` blocks the reservation however it is asked — three runs, three printouts to compare line by line. Then `--user priya` asking to reserve for daniel: the model writes what the message said, and the session decides. The caller, and so the filter string, was never the model's to choose.

Steps

1. Five copies, as shipped (`VALIDATE=off`). The schema says 3; watch what the model writes and who refuses it.

COMMAND — PASTE THIS LINE

```
python agent.py --user daniel "Reserve five copies of PRC-KYC-003 for daniel"
```

EXAMPLE

```
caller: daniel (Daniel Okafor, Dispute Officer, clearance confidential)
model: gemini-2.5-flash (Vertex AI, global)
api: <your BOOKSHELF_API_URL>
message: Reserve five copies of PRC-KYC-003 for daniel
→ model turn 1 tools=[lookup_stock, search_bookshelf, reserve_book] mode=AUTO grounding=off
→ model function_call reserve_book({"book_id": "PRC-KYC-003", "copies": 5, "caller": "daniel"})
⚙️ run reserve_book({"book_id": "PRC-KYC-003", "copies": 5, "caller": "daniel"}) → {"error": "HTTP 422 from
bookshelf-api: copies: Input should be between 1 and 3"}
→ function_response reserve_book {"error": "HTTP 422 from bookshelf-api: copies: Input should be between 1
and 3"}
→ model turn 2 tools=[lookup_stock, search_bookshelf, reserve_book] mode=AUTO grounding=off
→ model text: <the model reporting the API's error in its own words>
steps=2 calls=1 tokens=<n> cost=$<n> elapsed=<n>s
```

The model wrote `"copies": 5` — the schema is guidance to it, not a guarantee. The call was made (`calls=1`), `bookshelf-api` answered HTTP 422, and the error went back to the model. If your model wrote 3 instead, it corrected the request silently — write that down; it is a different problem.

2. The same request with `VALIDATE=on`: `agent.py` checks the arguments against the schema before running anything.

COMMAND — PASTE THIS LINE

```
VALIDATE=on python agent.py --user daniel "Reserve five copies of PRC-KYC-003 for daniel"
```

EXAMPLE

```
caller: daniel (Daniel Okafor, Dispute Officer, clearance confidential)
model: gemini-2.5-flash (Vertex AI, global)
api: <your BOOKSHELF_API_URL>
message: Reserve five copies of PRC-KYC-003 for daniel
→ model turn 1 tools=[lookup_stock, search_bookshelf, reserve_book] mode=AUTO grounding=off
→ model function_call reserve_book({"book_id": "PRC-KYC-003", "copies": 5, "caller": "daniel"})
⚙️ run reserve_book({"book_id": "PRC-KYC-003", "copies": 5, "caller": "daniel"}) → blocked: invalid args:
copies=5 > maximum 3
→ function_response reserve_book {"error": "invalid args: copies=5 > maximum 3"}
→ model turn 2 tools=[lookup_stock, search_bookshelf, reserve_book] mode=AUTO grounding=off
→ model text: <the model reporting the refusal>
steps=2 calls=0 tokens=<n> cost=$<n> elapsed=<n>s
```

Same `function_call`, and then ⚙️ `run reserve_book(...)` → `blocked: invalid args: copies=5 > maximum 3`. `calls=0`: nothing left the process. The same schema, read by your code instead of only by the model.

3. APPROVE=off: the reservation is blocked, however it is asked.

COMMAND — PASTE THIS LINE

```
APPROVE=off python agent.py --user daniel "Reserve one copy of PRC-KYC-003 for daniel"
```

EXAMPLE

```
caller: daniel (Daniel Okafor, Dispute Officer, clearance confidential)
model: gemini-2.5-flash (Vertex AI, global)
api: <your BOOKSHELF_API_URL>
message: Reserve one copy of PRC-KYC-003 for daniel
→ model turn 1 tools=[lookup_stock, search_bookshelf, reserve_book] mode=AUTO grounding=off
→ model function_call reserve_book({"book_id": "PRC-KYC-003", "copies": 1, "caller": "daniel"})
✱ run reserve_book({"book_id": "PRC-KYC-003", "copies": 1, "caller": "daniel"}) → blocked: needs approval
  (APPROVE=off)
→ function_response reserve_book {"error": "needs approval (APPROVE=off)"}
→ model turn 2 tools=[lookup_stock, search_bookshelf, reserve_book] mode=AUTO grounding=off
→ model text: <the model reporting that the reservation needs approval>
steps=2 calls=0 tokens=<n> cost=$<n> elapsed=<n>s
```

4. The same command twice more. Then put the three printouts side by side and compare them line by line: which lines are identical, and which are allowed to differ.

COMMAND — PASTE THIS LINE

```
APPROVE=off python agent.py --user daniel "Reserve one copy of PRC-KYC-003 for daniel"
```

COMMAND — PASTE THIS LINE

```
APPROVE=off python agent.py --user daniel "Reserve one copy of PRC-KYC-003 for daniel"
```

EXPECTED OUTPUT — THE LINES THAT ARE THE SAME ON ALL THREE RUNS

```
→ model function_call reserve_book({"book_id": "PRC-KYC-003", "copies": 1, "caller": "daniel"})
✱ run reserve_book({"book_id": "PRC-KYC-003", "copies": 1, "caller": "daniel"}) → blocked: needs approval
  (APPROVE=off)
→ function_response reserve_book {"error": "needs approval (APPROVE=off)"}
...
steps=2 calls=0 tokens=<n> cost=$<n> elapsed=<n>s
```

The `blocked:` line, the `function_response` and `calls=0` are identical on all three, because they were produced by code reading a switch. The model's `text:` line may be worded differently each time, and the elapsed time will differ; those were the model's to vary and the outcome was not.

5. priya asks to reserve for daniel. The model passes on what the message said; `agent.py` compares it with the session.

COMMAND — PASTE THIS LINE

```
python agent.py --user priya "Reserve one copy of PRC-KYC-003 for daniel"
```

EXAMPLE

```
caller: priya (Priya Nair, Customer Contact agent, clearance internal)
model: gemini-2.5-flash (Vertex AI, global)
api: <your BOOKSHELF_API_URL>
message: Reserve one copy of PRC-KYC-003 for daniel
→ model turn 1 tools=[lookup_stock, search_bookshelf, reserve_book] mode=AUTO grounding=off
→ model function_call reserve_book({"book_id": "PRC-KYC-003", "copies": 1, "caller": "daniel"})
✱ run reserve_book({"book_id": "PRC-KYC-003", "copies": 1, "caller": "daniel"}) → blocked: caller mismatch:
  model said 'daniel', this session is 'priya'
→ function_response reserve_book {"error": "caller must be priya"}
→ model turn 2 tools=[lookup_stock, search_bookshelf, reserve_book] mode=AUTO grounding=off
→ model text: <the model reporting that the caller must be priya>
steps=2 calls=0 tokens=<n> cost=$<n> elapsed=<n>s
```

`blocked: caller mismatch: model said 'daniel', this session is 'priya'.` The caller was decided by `--user` before the model saw anything — the same way the filter string was decided in Part B, and the same way `answer.sh --user` decided it yesterday. The `caller` argument exists so the model can say who it is acting for; the code decides whether to believe it.

✓ Check yourself

"Five copies": HTTP 422 with `calls=1` as shipped; blocked: `invalid args: copies=5 > maximum 3` with `calls=0` under `VALIDATE=on`.

Three `APPROVE=off` printouts with the same blocked: `needs approval (APPROVE=off)` line and `calls=0`.

`--user priya: blocked: caller mismatch`.

You can say, for each of the three decisions, where it was made — and that none of them was a sentence in the prompt.

Troubleshooting

Symptom	Cause	Fix
Step 1 was blocked with <code>invalid args</code>	<code>VALIDATE=on</code> is set in <code>agent.env</code> from an earlier edit	Set it to <code>off</code> in the file; type the switch in front of the command for step 2
Step 1: the model wrote <code>"copies": 3</code>	It corrected the request on its own	Note it — a silent correction is its own problem; step 2 still shows the check
Step 5 reserved	The model wrote <code>"caller": "priya"</code> despite the message	A legitimate outcome: the reservation is priya's, and the session decided. <code>--reset</code>
HTTP 409	The library is low	<code>python agent.py --reset</code>

Appendix A — tools.json, final (after Part C)

The three tools the model sees at the end of Lab 1: `lookup_stock` and `search_bookshelf` as shipped, and `reserve_book` as added in Part C. This is also `fixtures/tools.lab1c.json` and `fixtures/tools.lab1e-fixed.json`. Each entry is a name `agent.py` looks up in `tools_impl.py`, a description the model reads, and a JSON Schema the model shapes its arguments by and `VALIDATE=on` checks them against.

KIT FILE — TOOLS.JSON, FINAL (THE SHORT ARRAYS ON ONE LINE, TO FIT THE PAGE)

```
[
  {
    "name": "lookup_stock",
    "description": "Read-only: how many copies of one document in the Bookshelf library are available right now. Use for questions about stock or copies left.",
    "parameters": {
      "type": "object",
      "properties": {
        "book_id": {
          "type": "string",
          "description": "Document id, e.g. PRC-KYC-003"
        }
      },
      "required": ["book_id"]
    }
  },
  {
    "name": "search_bookshelf",
    "description": "Search the approved policies and procedures and answer a question from them, with citations. The caller's clearance decides which documents are searched.",
    "parameters": {
      "type": "object",
      "properties": {
        "question": {
          "type": "string",
          "description": "The question, in the caller's words"
        },
        "caller": {
          "type": "string",
          "enum": ["priya", "daniel", "mlro"],
          "description": "Who is asking"
        }
      },
      "required": ["question", "caller"]
    }
  },
  {
    "name": "reserve_book",
    "description": "Reserve copies of a document in the Bookshelf library for a member of staff. Use when the caller asks to reserve, hold or book copies. This changes library records.",
    "parameters": {
      "type": "object",
      "properties": {
        "book_id": {
          "type": "string",
          "description": "Document id, e.g. PRC-KYC-003"
        },
        "copies": {
          "type": "integer",
          "minimum": 1,
          "maximum": 3,
          "description": "Copies to reserve, 1 to 3"
        },
        "caller": {
          "type": "string",
          "enum": ["priya", "daniel", "mlro"],
          "description": "Who the reservation is for"
        }
      },
      "required": ["book_id", "copies", "caller"]
    }
  }
]
```

Appendix B — agent.env: the Lab 1 switches

The Lab 1 block of `agent.env.example`, verbatim. The values shown are the Part A state. A switch typed in front of the command (`MODE=NONE python agent.py ...`) wins over the file for that one run.

KIT FILE — AGENT.ENV.EXAMPLE, THE LAB 1 BLOCK

```
# ---- Lab 1
# A: no function tools are sent to the model. Part C turns this on.
TOOLS=off
# B: the Day 1 data store as a server-side retrieval tool, with the caller's filter string.
GROUNDING=off
# D: function_calling_config. MODE=AUTO | ANY | NONE. ALLOWED only counts when MODE=ANY.
MODE=AUTO
ALLOWED=
# D: on = run every function_call the model returns in a turn;
# off = run the first, answer {"error": "one action per turn"} to the rest.
PARALLEL=on
# F: check every call's arguments against the JSON Schema in code before running it.
VALIDATE=off
# F and Lab 2 D: reserve_book policy. off = blocked ("needs approval"), auto = runs,
# ask = print the approval card and wait for y/n. APPROVE_CARD=full | minimal.
APPROVE=auto
APPROVE_CARD=full
```

Switch	Values	Part	Effect
TOOLS	off / on	A, C	off : no function tools are sent; on : everything in <code>tools.json</code>
GROUNDING	off / on	B	on : the Day 1 data store as a server-side retrieval tool, with the caller's filter string
MODE	AUTO / ANY / NONE	D	function_calling_config : the model may call, must call, may not call. ANY applies to the first turn only
ALLOWED	tool names, comma-separated	D	With MODE=ANY : the forced call must be one of these. Ignored otherwise
PARALLEL	on / off	D	on : run every <code>function_call</code> in the turn; off : run the first, answer <code>{"error": "one action per turn"}</code> to the rest
VALIDATE	off / on	F	on : check each call's arguments against the JSON Schema in code before running it
APPROVE	auto / off / ask	F (and Day 3)	auto : <code>reserve_book</code> runs; off : blocked ("needs approval"); ask : an approval card and y/n — Day 3

What each part sets, in order: **A** as shipped · **B** `GROUNDING=on`, then `off` · **C** `TOOLS=on` · **D** `MODE`, `ALLOWED`, `PARALLEL` on the command line · **E** nothing (edits to `tools.json`) · **F** `VALIDATE`, `APPROVE` on the command line. Day 3 starts from `TOOLS=on`, `APPROVE=auto` and the rest as shipped.

Appendix C — The callers and their filter strings (callers.json)

Three callers, three clearances, three strings — the same file as Day 1. --user picks one; agent .py puts the caller's name and role in the model's instructions, sends the filter string inside the retrieval tool (Part B) and with the search_bookshelf call, and refuses a reserve_book call whose caller argument is not the session's (Part F). The model never writes a filter.

id	Name and role	Department	Clearance
priya	Priya Nair — Customer Contact agent	Customer Contact	internal
daniel	Daniel Okafor — Dispute Officer	Cards Operations	confidential
mlro	Sarah Whitmore — Deputy MLRO	Financial Crime Compliance	restricted

THE THREE STRINGS, VERBATIM

```
priya status: ANY("approved") AND classification: ANY("internal")
daniel status: ANY("approved") AND classification: ANY("internal","confidential")
mlro status: ANY("approved") AND classification: ANY("internal","confidential","restricted")
```

The file's own note: "One caller per clearance level. 'filter' is the exact Discovery Engine filter string answer.sh --user NAME and evaluate.py send. The filter is a metadata filter on indexable fields, not an access control: anyone who can call the API can send a different filter."